React – Fundamentals

Session – 5

1. Create a React component called LikeButton that displays a button and a count. When the button is clicked, increment the count and update the display.

Ans.

LikeButton.js:

```
import React, { useState } from "react";

function LikeButton() {
  const [count, setCount] = useState(0);

  const handleClick = () => {
    setCount(count + 1);
  };

  return (
    <div>
      <button onClick={handleClick}>👍 Like</button>
      <p>Likes: {count}</p>
    </div>
  );
}

export default LikeButton;
```

App.js:

```
import React from "react";
import LikeButton from "./LikeButton";

function App() {
  return (
    <div>
      <LikeButton />
    </div>
  );
}

export default App;
```

2. Build a SearchBar component where the user can type a product name (like Flipkart search). As the user types in the input box, display the current value below in real-time using React state.

Ans.

SearchBar.js:

```
import React, { useState } from "react";

function SearchBar() {
  const [search, setSearch] = useState("");

  return (
    <div>
      <h2>Flipkart Search</h2>

      <input
        type="text"
        placeholder="Search products..."
        value={search}
        onChange={(e) => setSearch(e.target.value)}
      />

      <p>You searched: {search}</p>
    </div>
  );
}

export default SearchBar;
```

App.js:

```
import React from "react";
import SearchBar from "./SearchBar";

function App() {
  return (
    <div>
      <SearchBar />
    </div>
  );
}

export default App;
```

3. Create a simple LoginForm component with two input fields: username and password. When the form is submitted, display an alert with the entered username

and password.

Hint: Use the onSubmit event on the form and prevent the default page reload.

Ans.

LoginForm.js:

```
import React, { useState } from "react";

function LoginForm() {
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();
    alert(`Username: ${username}\nPassword: ${password}`);
  };

  return (
    <form onSubmit={handleSubmit}>
      <h2>Login</h2>

      <input
        type="text"
        placeholder="Username"
        value={username}
        onChange={(e) => setUsername(e.target.value)}
      />
      <br /><br />

      <input
        type="password"
        placeholder="Password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />
      <br /><br />

      <button type="submit">Login</button>
    </form>
  );
}

export default LoginForm;
```

App.js:

```
import React from "react";
import LoginForm from "./LoginForm";
```

```
function App() {
  return (
    <div>
      <LoginForm />
    </div>
  );
}

export default App;
```

4. Build a PlaylistAdder component for a Spotify-style app: allow the user to enter a song name and artist, and when the form is submitted, add the song to a displayed list below the form.

Ans.

PlaylistAdder.js:

```
import React, { useState } from "react";

function PlaylistAdder() {
  const [song, setSong] = useState("");
  const [artist, setArtist] = useState("");
  const [playlist, setPlaylist] = useState([]);

  const handleSubmit = (e) => {
    e.preventDefault();

    const newSong = {
      song,
      artist,
    };

    setPlaylist([...playlist, newSong]);

    setSong("");
    setArtist("");
  };

  return (
    <div>
      <h2>Add Song</h2>

      <form onSubmit={handleSubmit}>
        <input
          type="text"
          placeholder="Song Name"
          value={song}
```

```
          onChange={(e) => setSong(e.target.value)}
        />
        <br /><br />

        <input
          type="text"
          placeholder="Artist"
          value={artist}
          onChange={(e) => setArtist(e.target.value)}
        />
        <br /><br />

        <button type="submit">Add Song</button>
      </form>

      <h3>Playlist</h3>

      <ul>
        {playlist.map((item, index) => (
          <li key={index}>
            {item.song} - {item.artist}
          </li>
        ))}
      </ul>
    </div>
  );
}

export default PlaylistAdder;
```

App.js:

```
import React from "react";
import PlaylistAdder from "./PlaylistAdder";

function App() {
  return (
    <div>
      <PlaylistAdder />
    </div>
  );
}

export default App;
```

5. Refactor your LoginForm to clear the input fields after successful form submission.

Constraint: Do not reload the page or use window.location.

Ans.

LoginForm.js:

```
import React, { useState } from "react";

function LoginForm() {
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");

  const handleSubmit = (e) => {
    e.preventDefault();

    alert(`Username: ${username}\nPassword: ${password}`);

    // Clear input fields
    setUsername("");
    setPassword("");
  };

  return (
    <form onSubmit={handleSubmit}>
      <h2>Login</h2>

      <input
        type="text"
        placeholder="Username"
        value={username}
        onChange={(e) => setUsername(e.target.value)}
      />
      <br /><br />

      <input
        type="password"
        placeholder="Password"
        value={password}
        onChange={(e) => setPassword(e.target.value)}
      />
      <br /><br />

      <button type="submit">Login</button>
    </form>
  );
}
```

```
export default LoginForm;
```

App.js:

```
import React from "react";
import LoginForm from "./LoginForm";

function App() {
  return (
    <div>
      <LoginForm />
    </div>
  );
}

export default App;
```